

```
import axios from 'axios';JavaScript  
  
const apiClient = axios.create({  
  baseURL: 'https://eu-central-1.hapio.net/v1/',  
  headers: {'Authorization': 'Bearer  
GsAnhHagT1lG9nniTAcwZWd4nUZXnGpAb9Ywyrxz'}  
});  
  
apiClient.put('services/' + service.id + '/resources/' +  
resource.id).then(function (response) {  
  const association = response.data;  
});
```

The response includes information about this association:

```
HTTP/2 201HTTP  
date: Thu, 24 Aug 2023 09:55:13 GMT  
content-type: application/json  
content-length: 213  
x-ratelimit-limit: 100  
x-ratelimit-remaining: 99  
access-control-allow-origin: *  
cache-control: no-cache, private  
apigw-requestid: KKNYPiW6FiAEPsw=  
  
{  
  "service_id": "a09c4585-0519-44f6-a5ab-647aeffc6281",  
  "resource_id": "c8924e50-af0d-47ae-bf5d-e184b1b59a60",  
  "created_at": "2023-08-24T09:55:13+00:00",  
  "updated_at": "2023-08-24T09:55:13+00:00"  
}
```

Set up a schedule

The last thing we need to do is to set up a schedule for our doctor. This will tell Hapio when Dr. Smith (our resource) is available for bookings at our clinic. Let's assume Dr. Smith's working hours are Monday through Friday, 08:00 to 17:00, with lunch between 12:00 and 13:00. For this kind of working hours, a recurring schedule is the best option, since this lets us set up a schedule that repeats every week.

We start by creating a recurring schedule by send a **POST** request to the endpoint [/v1/resources/{resource ID}/recurring-schedules](#):

```
POST /v1/resources/c8924e50-af0d-47ae-bf5d-e184b1b59a60/ HTTP
recurring-schedules
HTTP/2
Host: eu-central-1.hapio.net
Accept: */*
Authorization: Bearer GsAnhHagTllG9nniTAcwZWd4nUZXnGpAb9Ywyrxz
Content-Type: application/json
Content-Length: 93

{
  "location_id": "2b113cf6-5d6c-4a31-bd21-88ba7fb440ea",
  "start_date": "2023-08-24"
}
```

```
use Hapio\Sdk\ApiClient;
use Hapio\Sdk\Models\RecurringSchedule;

$apiClient = new ApiClient('GsAnhHagTllG9nniTAcwZWd4nUZXnGpAb9Ywyrxz');

$recurringSchedule = new RecurringSchedule();
$recurringSchedule->locationId = $location->id;
$recurringSchedule->start_date = '2023-08-24';

$recurringSchedule = $apiClient->recurringSchedules()->store(
    [$resource->id],
    $recurringSchedule
);
```

```
import axios from 'axios';

const apiClient = axios.create({
  baseURL: 'https://eu-central-1.hapio.net/v1/',
  headers: {'Authorization': 'Bearer GsAnhHagTllG9nniTAcwZWd4nUZXnGpAb9Ywyrxz'}
});
```

```
apiClient.post('resources/' + resource.id + '/recurring-schedules', {
  location_id: location.id,
  start_date: '2023-08-24'
}).then(function (response) {
  const recurringSchedule = response.data;
});
```

As you can see, the request includes the ID of our location, since every recurring schedule belongs to a specific location. As usual, the response include all information about the recurring schedule that we just created, including the location that it belongs to:

```
HTTP/2 201 HTTP
date: Thu, 24 Aug 2023 10:20:42 GMT
content-type: application/json
content-length: 605
x-ratelimit-limit: 100
x-ratelimit-remaining: 99
access-control-allow-origin: *
cache-control: no-cache, private
apigw-requestid: KKRHFIBLFiAEM8g=

{
  "id": "e518071e-b4d6-4e98-a12f-04b44276e93c",
  "location": {
    "id": "2b113cf6-5d6c-4a31-bd21-88ba7fb440ea",
    "name": "Perfect Health - Stockholm",
    "time_zone": "Europe/Stockholm",
    "resource_selection_strategy": "equalize",
    "metadata": null,
    "protected_metadata": null,
    "enabled": true,
    "created_at": "2023-08-24T09:42:13+00:00",
    "updated_at": "2023-08-24T09:42:13+00:00"
  },
  "start_date": "2023-08-24",
  "end_date": null,
  "updated_at": "2023-08-24T10:20:42+00:00",
  "created_at": "2023-08-24T10:20:42+00:00"
}
```

Now, we can create schedule blocks in this recurring schedule. This is done by sending **POST** requests to the endpoint </v1/resources/{resource ID}/recurring-schedules/{recurring schedule ID}/schedule-blocks>:

```

POST                                                                    HTTP
/v1/resources/c8924e50-af0d-47ae-bf5d-e184b1b59a60/recurring-
schedules/e518071e-b4d6-4e98-a12f-04b44276e93c/schedule-blocks HTTP/2
Host: eu-central-1.hapio.net
Accept: */*
Authorization: Bearer GsAnhHagT1lG9nniTAcwZWd4nUZXnGpAb9Ywyrxz
Content-Type: application/json
Content-Length: 85

{
  "weekday": "monday",
  "start_time": "08:00:00",
  "end_time": "12:00:00"
}
```

```

use Hapio\Sdk\ApiClient;                                                                    PHP
use Hapio\Sdk\Models\RecurringScheduleBlock;

$apiClient = new ApiClient('GsAnhHagT1lG9nniTAcwZWd4nUZXnGpAb9Ywyrxz');

$recurringScheduleBlock = new RecurringScheduleBlock();
$recurringScheduleBlock->weekday = 'monday';
$recurringScheduleBlock->start_time = '08:00:00';
$recurringScheduleBlock->end_time = '12:00:00';

$recurringScheduleBlock = $apiClient->recurringScheduleBlocks()->store(
    [$resource->id, $recurringSchedule->id],
    $recurringScheduleBlock
);
```

```

import axios from 'axios';                                                                    JavaScript

const apiClient = axios.create({
  baseURL: 'https://eu-central-1.hapio.net/v1/',
  headers: {'Authorization': 'Bearer
GsAnhHagT1lG9nniTAcwZWd4nUZXnGpAb9Ywyrxz'}}
});
```

```
apiClient.post('resources/' + resource.id + '/recurring-schedules/' +  
recurringSchedule.id + '/schedule-blocks', {  
  weekday: 'monday',  
  start_time: '08:00:00',  
  end_time: '12:00:00'  
}).then(function (response) {  
  const recurringScheduleBlock = response.data;  
});
```

As you can see, our first schedule block in this recurring schedule is for Mondays from 08:00 to 12:00. The schedule block ends at 12:00, since that's when Dr. Smith's lunch starts, and the next schedule block should then start at 13:00. The response will, as usual, include all information about the created recurring schedule block:

```
HTTP/2 201 HTTP  
date: Thu, 24 Aug 2023 10:28:22 GMT  
content-type: application/json  
content-length: 229  
access-control-allow-origin: *  
cache-control: no-cache, private  
x-ratelimit-limit: 100  
x-ratelimit-remaining: 99  
apigw-requestid: KKSPAj9rFiAEP6A=  
  
{  
  "id": "456da208-72d4-4ac0-b11c-324f4b5bcd34",  
  "weekday": "monday",  
  "start_time": "08:00:00",  
  "end_time": "12:00:00",  
  "created_at": "2023-08-24T10:28:22+00:00",  
  "updated_at": "2023-08-24T10:28:22+00:00"  
}
```

By continuing to create similar schedule blocks, we can fill up the entire week so that it reflects Dr. Smith's working hours.

Get bookable slots

Once we have our recurring schedule for the resource set up, we can retrieve bookable slots for the service that we created. We do this by sending a **GET** request to the endpoint [/v1/services/{service ID}/bookable-slots](#):

GET

HTTP

```
/v1/services/a09c4585-0519-44f6-a5ab-647aeffc6281/bookable-slots?
from=2023-08-28T08:00:00%2B02:00&to=2023-08-28T10:00:00%2B02:00&loca
tion=2b113cf6-5d6c-4a31-bd21-88ba7fb440ea HTTP/2
Host: eu-central-1.hapio.net
Accept: */*
Authorization: Bearer GsAnhHagT1lG9nniTAcwZWd4nUZXnGpAb9Ywyrxz
```

```
use Hapio\Sdk\ApiClient;
```

PHP

```
$apiClient = new ApiClient('GsAnhHagT1lG9nniTAcwZWd4nUZXnGpAb9Ywyrxz');

$page = $apiClient->services()->listBookableSlots(
    $service->id,
    [
        'from' => new DateTime('2023-08-28 08:00:00+02:00'),
        'to' => new DateTime('2023-08-28 10:00:00+02:00'),
        'location' => $location->id,
    ]
);
```

```
import axios from 'axios';
```

JavaScript

```
const apiClient = axios.create({
  baseURL: 'https://eu-central-1.hapio.net/v1/',
  headers: {'Authorization': 'Bearer GsAnhHagT1lG9nniTAcwZWd4nUZXnGpAb9Ywyrxz'}
});

apiClient.get('services/' + service.id + '/bookable-slots', {
  params: {
    from: '2023-08-28T08:00:00+02:00',
    to: '2023-08-28T10:00:00+02:00',
    location: location.id
  }
});
```

```
}).then(function (response) {  
    const page = response.data;  
});
```

Note that we need to include a time window and a location ID in the query string of the request, since Hapio needs that information to determine when and where to look for bookable slots. It's important to remember to URL encode any special characters in the query string, such as the plus sign between the timestamp and the time zone offset.

The response includes a paginated list of bookable slots during the time window and at the location that was requested. In this example, the time window was from 08:00 to 10:00 on a monday, and since we have defined a schedule that is open during this time on mondays, we get two bookable slots, 08:00 to 08:50, and 09:00 to 09:50.

```
HTTP/2 200 HTTP  
date: Thu, 24 Aug 2023 14:19:07 GMT  
content-type: application/json  
content-length: 2302  
cache-control: no-cache, private  
x-ratelimit-limit: 100  
x-ratelimit-remaining: 99  
access-control-allow-origin: *  
apigw-requestid: KK0COjkrFiAEMjw=  
  
{  
  "data": [  
    {  
      "buffer_starts_at": "2023-08-28T08:00:00+02:00",  
      "starts_at": "2023-08-28T08:00:00+02:00",  
      "ends_at": "2023-08-28T08:50:00+02:00",  
      "buffer_ends_at": "2023-08-28T09:00:00+02:00",  
      "resources": [  
        {  
          "id": "c8924e50-af0d-47ae-bf5d-e184b1b59a60",  
          "name": "Dr. Smith",  
          "max_simultaneous_bookings": 1,  
          "metadata": null,  
          "protected_metadata": null,  
          "enabled": true,  
          "created_at": "2023-08-24T08:58:04+00:00",  
          "updated_at": "2023-08-24T08:58:04+00:00"  
        }  
      ]  
    }  
  ]  
}
```

```

    ]
  },
  {
    "buffer_starts_at": "2023-08-28T09:00:00+02:00",
    "starts_at": "2023-08-28T09:00:00+02:00",
    "ends_at": "2023-08-28T09:50:00+02:00",
    "buffer_ends_at": "2023-08-28T10:00:00+02:00",
    "resources": [
      {
        "id": "c8924e50-af0d-47ae-bf5d-e184b1b59a60",
        "name": "Dr. Smith",
        "max_simultaneous_bookings": 1,
        "metadata": null,
        "protected_metadata": null,
        "enabled": true,
        "created_at": "2023-08-24T08:58:04+00:00",
        "updated_at": "2023-08-24T08:58:04+00:00"
      }
    ]
  }
],
"links": {
  "first": "https://eu-central-1.hapio.net/v1/services/a09c4585-0519-44f6-a5ab-647aeffc6281/bookable-slots?from=2023-08-28T08%3A00%3A00%2B02%3A00&location=2b113cf6-5d6c-4a31-bd21-88ba7fb440ea&to=2023-08-28T10%3A00%3A00%2B02%3A00&page=1",
  "last": "https://eu-central-1.hapio.net/v1/services/a09c4585-0519-44f6-a5ab-647aeffc6281/bookable-slots?from=2023-08-28T08%3A00%3A00%2B02%3A00&location=2b113cf6-5d6c-4a31-bd21-88ba7fb440ea&to=2023-08-28T10%3A00%3A00%2B02%3A00&page=1",
  "prev": null,
  "next": null
},
"meta": {
  "current_page": 1,
  "from": 1,
  "last_page": 1,
  "path": "https://eu-central-1.hapio.net/v1/services/a09c4585-0519-44f6-a5ab-647aeffc6281/bookable-slots",
  "per_page": 100,
  "to": 2,
  "total": 2
}
}

```


Create your first booking

Now that everything is set up, and you have made a request to fetch bookable slots in order to find out when the doctor is available, you are ready to create your first booking. Do this by sending a **POST** request to the endpoint </v1/bookings>:

```
POST /v1/bookings HTTP/2
Host: eu-central-1.hapio.net
Accept: */*
Authorization: Bearer GsAnhHagTlIG9nniTAcwZWd4nUZXnGpAb9Ywyrxz
Content-Type: application/json
Content-Length: 209

{
  "location_id": "2b113cf6-5d6c-4a31-bd21-88ba7fb440ea",
  "service_id": "a09c4585-0519-44f6-a5ab-647aefc6281",
  "starts_at": "2023-08-28T08:00:00+02:00",
  "ends_at": "2023-08-28T08:50:00+02:00"
}
```

```
use Hapio\Sdk\ApiClient;
use Hapio\Sdk\Models\Booking;

$apiClient = new ApiClient('GsAnhHagTlIG9nniTAcwZWd4nUZXnGpAb9Ywyrxz');

$booking = new Booking();
$booking->locationId = $location->id;
$booking->serviceId = $service->id;
$booking->startsAt = new DateTime('2023-08-28 08:00:00+02:00');
$booking->endsAt = new DateTime('2023-08-28 08:50:00+02:00');

$booking = $apiClient->bookings()->store($booking);
```

```
import axios from 'axios';

const apiClient = axios.create({
  baseURL: 'https://eu-central-1.hapio.net/v1/',
  headers: {'Authorization': 'Bearer GsAnhHagTlIG9nniTAcwZWd4nUZXnGpAb9Ywyrxz'}
});
```

```
apiClient.post('bookings', {
  location_id: location.id,
  service_id: service.id,
  starts_at: '2023-08-28T08:00:00+02:00',
  ends_at: '2023-08-28T08:50:00+02:00'
}).then(function (response) {
  const booking = response.data;
});
```

In this case, we only include the minimum amount of information needed to create a booking, namely the location, the service, and the start and end timestamps. We let Hapio automatically fill in the rest, including the resource (according to the resource selection strategy of the location).

As usual, the response include all information about the booking that you've just created:

```
HTTP/2 201 HTTP
date: Thu, 24 Aug 2023 14:36:20 GMT
content-type: application/json
content-length: 1900
cache-control: no-cache, private
x-ratelimit-limit: 100
x-ratelimit-remaining: 99
access-control-allow-origin: *
apigw-requestid: KK2jphEuliAEM4A=

{
  "id": "b3f06e22-2e2e-44a4-a9e5-7ed24b0a06a1",
  "resource": {
    "id": "c8924e50-af0d-47ae-bf5d-e184b1b59a60",
    "name": "Dr. Smith",
    "max_simultaneous_bookings": 1,
    "metadata": null,
    "protected_metadata": null,
    "enabled": true,
    "created_at": "2023-08-24T08:58:04+00:00",
    "updated_at": "2023-08-24T08:58:04+00:00"
  },
  "service": {
    "id": "a09c4585-0519-44f6-a5ab-647aeffc6281",
    "name": "Physical Exam",
    "price": "149.000",
```

```
    "type": "fixed",
    "duration": "PT50M",
    "bookable_interval": "PT1H",
    "buffer_time_before": "PT0S",
    "buffer_time_after": "PT10M",
    "booking_window_start": "PT2H",
    "booking_window_end": "P14D",
    "cancellation_threshold": "PT12H",
    "metadata": null,
    "protected_metadata": null,
    "enabled": true,
    "created_at": "2023-08-24T09:07:53+00:00",
    "updated_at": "2023-08-24T09:07:53+00:00"
  },
  "location": {
    "id": "2b113cf6-5d6c-4a31-bd21-88ba7fb440ea",
    "name": "Perfect Health - Stockholm",
    "time_zone": "Europe/Stockholm",
    "resource_selection_strategy": "equalize",
    "metadata": null,
    "protected_metadata": null,
    "enabled": true,
    "created_at": "2023-08-24T09:42:13+00:00",
    "updated_at": "2023-08-24T09:42:13+00:00"
  },
  "price": "149.000",
  "metadata": null,
  "protected_metadata": null,
  "is_temporary": false,
  "is_canceled": false,
  "starts_at": "2023-08-28T08:00:00+02:00",
  "ends_at": "2023-08-28T08:50:00+02:00",
  "buffer_starts_at": "2023-08-28T08:00:00+02:00",
  "buffer_ends_at": "2023-08-28T09:00:00+02:00",
  "created_at": "2023-08-24T14:36:20+00:00",
  "updated_at": "2023-08-24T14:36:20+00:00",
  "finalized_at": "2023-08-24T14:36:20+00:00",
  "canceled_at": null
}
```

That's it! You have now gone through all the necessary steps to create your first booking in Hapio.